COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/29/26	**DATE SUBMITTED:**	07/29/26
NAME:	Macasaet John David C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO.___ HOA 3.1 Linked List

1. PROCEDURE OUTPUT

Screenshot	mainsingly_ll.cpp > main() 1 #include <iostream> 2 #include "singly_ll.h" 3 4 int main() { 5 // CREATING A NODE STORED IN THE STACK 6 SingleList<char>* head = nullptr; 7 SingleList<char>* second = nullptr; 8 SingleList<char>* third = nullptr; 9 SingleList<char>* fourth = nullptr; 10 SingleList<char>* fifth = nullptr; 11 // ALLOCATES MEMORY 12 head = new SingleList<char>; 13 second = new SingleList<char>; 14 third = new SingleList<char>; 15 fourth = new SingleList<char>; 16 fifth = new SingleList<char>; 17 head->data = 'S'; 18 second->data = 'H'; 19 third->data = 'I'; 20 fourth->data = 'N'; 21 fifth->data = 'G'; 22 // LINKING THE NODES: 23 head->next = second; 24 second->next = third; 25 third->next = fourth; 26 fourth->next = fifth; 27 fifth->next = nullptr;
Discussion	In this you initialize the head and the rest of the nodes. We first create the nodes at the start and put values in them by the use of '->data'. In this we also linked the nodes.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**

Traversal

```cpp
template <typename T>
void ListTraversal(SingleList<T>* head){

    if (head == nullptr){
        std::cout << "The list is empty";
        return;
    }

    while(head != nullptr){
        // PRINT data OF n
        std::cout<<head->data;

        if(head->next != nullptr){
            std::cout << "->";
        }

        // GO TO NEXT NODE n -> next
        head = head->next;
    }

    std::cout << std::endl;
}
```

Insertion at the head

```cpp
template <typename T>
void sllinsertHead(T newData, SingleList<T>** head){

    // 1. Allocate new memory for the new code
    SingleList<T>* newNode = new SingleList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set next to the new node to point to the previous head
    newNode->next = *head;
    // 4. Reset head to the new node
    *head = newNode;
}
```

Insertion at any part of the list

```cpp
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){

    //1. Check if it is the head node (previous node is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //3. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;
    //4. Store data in the new node
    newNode->data = newData;
    //5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    //6. Point previous node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end

```cpp
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to null
    newNode->next = nullptr;

    // 2. Dereference the head node
    SingleList<T>* currentNode = *head;

    // 5. Traverse the list
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node

```cpp
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){
    // if there is nothing to delete
    if (*head == nullptr) return;
    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }
    // if data not found currNode == nullPtr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }
    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2. Code for the List Operations

A. SOURCE CODE	`std::cout << "Testing of traversal\n";` `ListTraversal(head);`

A. CONSOLE	`Testing of traversal`
	`S->H->I->N->G`
	For this we used the ListTraversal() function to traverse the list starting from the head

B. SOURCE CODE	`std::cout << "Testing of sllinsertHead\n";`
	`sllinsertHead('Q', &head);`
	`ListTraversal(head);`

B. CONSOLE	`Testing of sllinsertHead`
	`Q->S->H->I->N->G`
	We insert Q at the start and assign the address of head to Q.

C. SOURCE CODE	`std::cout << "Testing of sllGeneralInsert\n";`
	`sllGeneralInsert('X', head->next->next);`
	`ListTraversal(head);`

C. CONSOLE	`Testing of sllGeneralInsert`
	`Q->S->H->X->I->N->G`
	We insert X after head->next->next which is H. We insert X in the middle of H and I.

D. SOURCE CODE	`std::cout << "Testing of sllinsertEnd\n";`
	`sllinsertEnd('U', &head);`
	`ListTraversal(head);`

D. CONSOLE	`Testing of sllinsertEnd`
	`Q->S->H->X->I->N->G->U`
	We insert U at the end of the list and we traverse the list first until we

	reach the end before we insert. In the linked list you always start at the head.

E. SOURCE CODE	`std::cout << "Testing of deleting the node\n"`
	`sllDeleteNode('U', &head);`
	`ListTraversal(head);`

E. CONSOLE	`Testing of deleting the node`
	`Q->S->H->X->I->N->G`
	We delete the U node at the end of the linked list

F. SOURCE CODE	`std::cout << "Deleting all nodes\n";`
	`sllDeleteList(&head);`
	`ListTraversal(head);`

F. CONSOLE	`Deleting all nodes`
	`The list is empty`
	We delete all nodes starting from the head. If all was deleted a message will be sent saying the list is empty.

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis

```cpp
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){

    DoubleList<T> *tail; //temp store last node visited
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }
    std::cout<<"Forward: \n";
    //move forward
    while(currentNode != nullptr){
        std::cout<< currentNode->data<<" ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    //add a next line
    std::cout<<std::endl;
    //move backward
    std::cout<<"Back: \n";
    while(tail != nullptr){
        std::cout<< tail->data<<" ";
        tail = tail->prev;
    }
}
```

```
Testing the DLL Traversal
Forward:
C P E
Back:
E P C
```

We added a way to move backwards and we did this by storing the final node in another variable tail and starting from tail we traverse backwards.

```cpp
//insertion at the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T> ** currentHead){
    //creating a new node
    DoubleList<T>* newNode = CreateNewNode(newData);

    //new node should point to the current head
    newNode->next = *currentHead;
    //current head should point back to the new node
    (*currentHead)->prev = newNode;
    //update pointer head
    *currentHead = newNode;
}
```

```
Testing the insertion at the head node
Forward:
X C P E
Back:
E P C X
```

The currentHead points back to newNode so that the X will show up when we traverse backwards.

```
template <typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead){
    //create a new node
    DoubleList<T>* newNode  = CreateNewNode(newData);
    //traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    //connect the last node to the new node
    currentHead->next  = newNode;

    //connect the new node back to the last node
    newNode->prev = currentHead;
}
```

```
Testing the insertion at the end
Forward:
X C P E 0
Back:
0 E P C X
```

While using a while loop to traverse until we reach the end. Once we reach the end we assign newNode to currentHead->next so that the newNode will be attached at the end of the list. Then we assign currentHead to newNode->prev to make backwards traversing possible.

```
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    DoubleList<T>* newNode  = CreateNewNode(newData);
    DoubleList<T>* nextNode;

    newNode->data = newData;
    nextNode = prevNode->next;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
    nextNode->prev = newNode;
    newNode->prev = prevNode;

}
```

```
Testing of dllGeneralInsert
Forward:
X C J P E 0
Back:
0 E P J C X
```

We needed to create a node called nextNode to store prevNode->next. This will make it possible for our inserted node to show in backwards traversing. In our output we store C in nextNode and we point nextNode->prev to newNode(J).

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){
    // if there is nothing to delete
    if (*head == nullptr) return;
    DoubleList<T>* currNode = *head;
    DoubleList<T>* prevNode = nullptr;
    DoubleList<T>* nodeNext;
    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }
    nodeNext = currNode->next;
    // if data not found currNode == nullPtr in the while loop
    if(currNode == nullptr){
        std::cout << "the data was not found\n" << std::endl;
        return;
    }

    // if data was found
    if(prevNode == nullptr){
        *head = currNode->next;
        nodeNext = currNode;
    } else if (nodeNext != nullptr) {
        prevNode->next = currNode->next;
        nodeNext->prev = currNode->prev;
    }
    delete currNode;
```

```
Testing of dllDeleteNode
Forward:
X J P E 0
Back:
0 E P J X
```

In this we delete C, we did this by creating a new node named nodeNext just like how we did in General Insert to make C not show up when we traverse backwards.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

C supplesingly_ll.h > deleteSong<T>(T, musicList<T>**)

```h
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class musicList{
    public:
        T data;
        musicList<T>* prev = nullptr;
        musicList<T>* next = nullptr;
};

//creating a node:
template <typename T>
musicList<T> *AddSong(T newData){
    //dynamically allocate a new memory for the node
    musicList<T> *newNode = new musicList<T>;
    //store the data in the node
    newNode->data = newData;
    //point next and prev to null
    newNode->next = nullptr;
    newNode->prev = nullptr;
    //return the address of the newly created node

    return newNode;
}

template <typename T>
void playlistTraverse(musicList<T>* currentNode){

```

```
    musicList<T> *headNode; //temp store last node visited
    //check if the list is empty
    if(currentNode == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }
    headNode = currentNode;
    //move forward
    std::cout<<"Music Playlist"<<std::endl;
    while(currentNode != nullptr){
        std::cout<< currentNode->data<<" \n";
        currentNode = currentNode->next;
    }
    std::cout<<":End of Playlist:"<<std::endl;
    if(currentNode == nullptr){
        std::cout<< "--(Return to Start)--\n"<< headNode->data<<"\n";
    }
}

template <typename T>
void playlistInsertEnd(T newData, musicList<T>* currentHead){
    //create a new node
    musicList<T>* newNode  = AddSong(newData);
    //traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }

    //connect the last node to the new node
    currentHead->next  = newNode;

    //connect the new node back to the last node
    newNode->prev = currentHead;
}

template <typename T>
void deleteSong(T findData, musicList<T>** head){

    // if there is nothing to delete
    if (*head == nullptr) return;

    musicList<T>* currNode = *head;
    musicList<T>* prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }

    // if data not found currNode == nullPtr in the while loop
```

```
82      if(currNode == nullptr){
83          std::cout << "the data was not found\n" << std::endl;
84          return;
85      }
86
87      // if data was found
88      if(prevNode == nullptr){
89          *head = currNode->next;
90      } else {
91          prevNode->next = currNode->next;
92      }
93
94      delete currNode;
95  }
96
97  #endif
```

CPP FILE:

supplementary.cpp > ...

```
1   #include <iostream>
2   #include "supplesingly_ll.h"
3   #include <string>
4
5
6   int main(){
7       // create a playlist
8       musicList<std::string>* head = AddSong(std::string("Linger - The Cranberries"));
9       musicList<std::string>* second = AddSong(std::string("Boys Don't Cry - The Cure"));
10      musicList<std::string>* third = AddSong(std::string("Let Down - Radiohead"));
11      musicList<std::string>* fourth = AddSong(std::string("Playing God - Paramore"));
12      musicList<std::string>* fifth = AddSong(std::string("Lovers Rock - TV Girl"));
13
14      head->next = second;
15      second->prev = head;
16      second->next = third;
17      third->prev = second;
18      third->next = fourth;
19      fourth->prev = third;
20      fourth->next = fifth;
21      fifth->prev = fourth;
22
23      int options = 0;
24      while(options <= 3){
25       std::cout<<"Menu:\n1. Start the Playlist\n2. Insert Song\n3. Remove Song\n4. Exit\n";
26       std::cout<<"Type the number of the option you want: ";
27       std::cin>>options;
28       switch(options){
```

```
29          case 1:{
30              //traversal
31              playlistTraverse(head);
32              std::cout<<std::endl;
33              break;
34          }
35          case 2:{
36              std::cout<<"Song Name: ";
37              std::string insertSong;
38              std::cin>>std::ws;
39              std::cout<<std::endl;
40              std::getline(std::cin, insertSong);
41              playlistInsertEnd(insertSong, head);
42              break;
43          }
44          case 3:{
45              std::cout<<"Song Name: ";
46              std::string removeSong;
47              std::cin>>std::ws;
48              std::cout<<std::endl;
49              std::getline(std::cin, removeSong);
50              deleteSong(removeSong, &head);
51              break;
52           }
53           case 4:{
54              break;
55           }
56      }
57  }
```

Start playlist:

```
Menu:
1. Start the Playlist
2. Insert Song
3. Remove Song
4. Exit
Type the number of the option you want: 1
Music Playlist
Linger - The Cranberries
Boys Don't Cry - The Cure
Let Down - Radiohead
Playing God - Paramore
Lovers Rock - TV Girl
:End of Playlist:
--(Return to Start)--
Linger - The Cranberries
```

Insert Song:

```
Menu:
1. Start the Playlist
2. Insert Song
3. Remove Song
4. Exit
Type the number of the option you want: 2
Song Name: Pare Ko - Eraserheads

Menu:
1. Start the Playlist
2. Insert Song
3. Remove Song
4. Exit
Type the number of the option you want: 1
Music Playlist
Linger - The Cranberries
Boys Don't Cry - The Cure
Let Down - Radiohead
Playing God - Paramore
Lovers Rock - TV Girl
Pare Ko - Eraserheads
:End of Playlist:
--(Return to Start)--
Linger - The Cranberries
```

Remove Song:

```
Menu:
1. Start the Playlist
2. Insert Song
3. Remove Song
4. Exit
Type the number of the option you want: 3
Song Name: Playing God - Paramore

Menu:
1. Start the Playlist
2. Insert Song
3. Remove Song
4. Exit
Type the number of the option you want: 1
Music Playlist
Linger - The Cranberries
Boys Don't Cry - The Cure
Let Down - Radiohead
Lovers Rock - TV Girl
Pare Ko - Eraserheads
:End of Playlist:
--(Return to Start)--
Linger - The Cranberries
```

CONCLUSION:
In this activity I gained a deeper understanding on linked lists, both singly and doubly linked lists. I learned the difference between this two and how in doubly you traverse back. We also made a playlist using a linked list, we used it to point back to the start song once the playlist ends.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:
Google Gemini

Examples:

Purpose of Use:

Examples: Debugging

Extent of Use: **When I don't know what to fix with the syntax for example that causes errors, I ask AI once I give up and I can't figure out what is wrong.**

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.